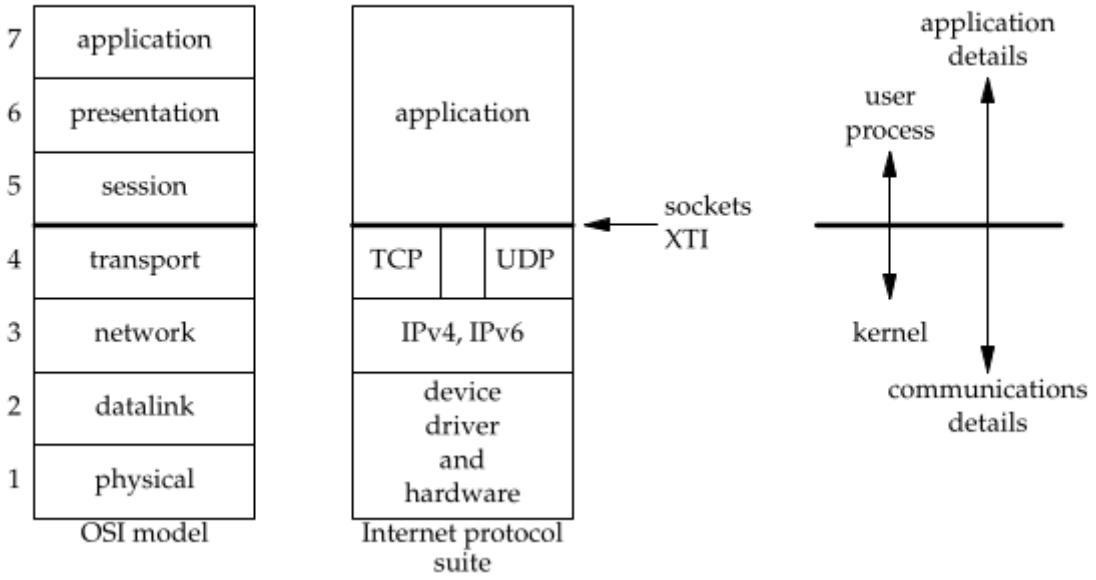
[Huyen Nguyen] - Socket Programming - TCP/UDP Sockets & I/O Multiplexing Socket programming

1.1. OSI Model



Where does the Socket API sit in the network model? Why is it placed there?

Consolidating Layers 5, 6, and 7: These three layers are merged into a single layer called the Application Layer (which represents programs like Web clients, Web servers, etc.).

Why is the Socket API located between Layer 4 and Layer 5?

Separating the User Process and OS Kernel Boundaries:

- The Application layer runs in User Space. If an application crashes, it does not crash the entire operating system.
- The bottom four layers reside inside the OS Kernel to prevent user-space applications from arbitrarily tampering with or disrupting the shared network transmission path.

=> The Socket API enables a standard User Process to instruct the complex protocol stack inside the OS Kernel to send and receive network data securely and easily, without needing deep knowledge of the underlying network hardware

1.2. BSD Networking History

4.2BSD (1983): Introduced the Socket API (socket, bind, connect, listen, accept).

4.3BSD Tahoe (1988): Implemented Slow Start, Congestion Avoidance, and Fast Retransmit algorithms.

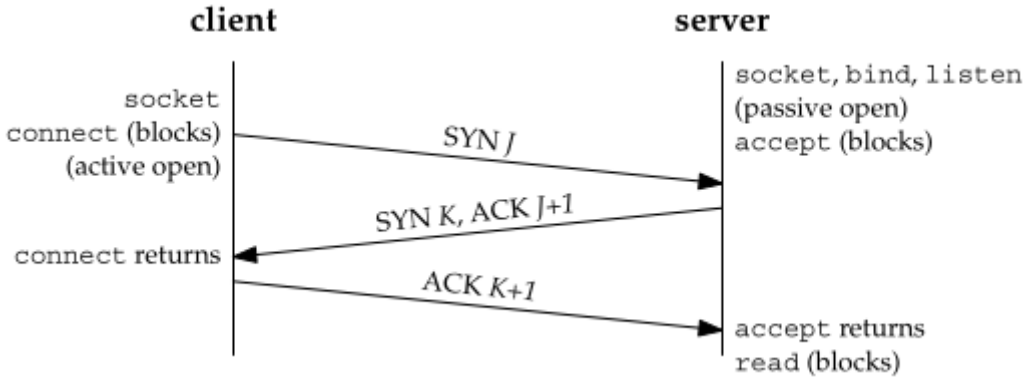
4.3BSD Reno (1990): Added the Fast Recovery algorithm, optimized routing tables, and updated the sockaddr structure.

4.4BSD (1993): Added support for IP Multicast and optimized performance for Long Fat Networks (LFNs).

2. Transport Layer theory

2.1. TCP Connection Establishment and Termination

Three-Way Handshake



When the Client calls connect() and the Server calls accept(), how are the underlying network packets actually transmitted back and forth?

Preparation

- The Server calls socket(), bind(), and listen() to get ready for incoming Client connections (Passive Open).
- The Server calls the accept() function, which blocks (waits) until a Client initiates a connection.

Client Initiates Connection

- The Client calls the connect() function (Active Open). The connect() function blocks (does not return immediately).
- The Client's OS Kernel automatically transmits a SYN packet to the Server: SYN (Synchronize) contains an initial sequence number randomly generated by the Client, denoted as J. This packet carries no payload data, consisting only of IP and TCP headers.

Server Responds

Upon receiving SYN J, the Server sends a single response packet back to the Client containing two distinct flags:

- ACK J+1: Acknowledges receipt of the Client's SYN (By convention, a SYN packet consumes 1 sequence number; thus, ACK equals J + 1, signifying "I have received J, please send starting from J+1 next time").
- SYN K: The Server transmits its own randomly generated initial sequence number (denoted as K) for the Client to synchronize.

Client Finalizes Connection

- The Client receives packet 2.
- The Client's connect() function unblocks and returns successfully (the Client is now ready to transmit data).
- The Client transmits the final packet: ACK K+1 to acknowledge receipt of the Server's SYN.
- Upon receiving this packet on the Server side, the Server's accept() function unblocks and returns successfully (returning a new connected socket descriptor, connfd). Both endpoints have officially established a network connection.

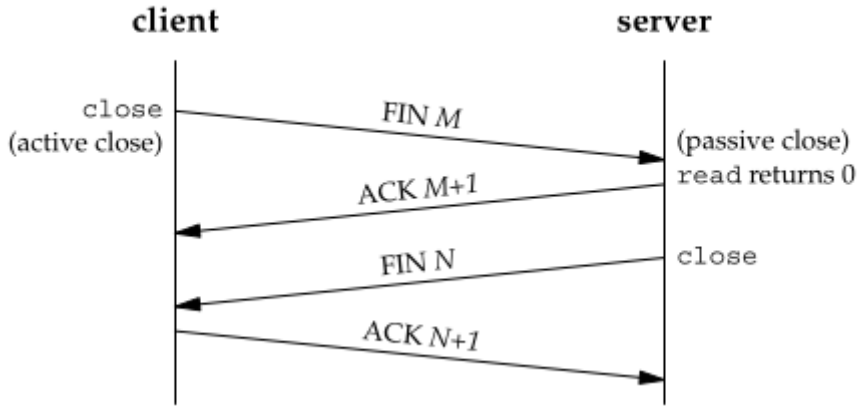
TCP Connection Termination - 4-Way Handshake

TCP connections are Full-Duplex, meaning there are two independent data streams flowing in opposite directions:

- Outbound: Client -> Server
- Inbound: Server -> Client

To close a connection, each direction must be terminated independently using a pair of packets: FIN (Finish – no more data) and ACK (Acknowledge – received notification). Two directions times 2

packets = 4 Packets in total



FIN M Packet

- The Client calls close(). The OS kernel sends a FIN packet with sequence number M to the Server, carrying the message: "I have sent all my data; no more bytes will be transmitted."
- The side initiating the close() call first is referred to as the Active Close party.

ACK M+1 Packet

The Server receives FIN M. The Server's OS kernel automatically responds with an ACK M+1 packet: "Acknowledged that you will not send any more data."

-> The read() function waiting on the Server immediately returns 0 (EOF – End of File).

Half-Close State:

- The Client -> Server stream is now closed.
- However, the Server -> Client stream remains open. If the Server still has pending data in its buffer, it retains the right to transmit the remaining data to the Client.

FIN N Packet

- After finishing the transmission of all remaining data, the Server officially calls its own close() function.
- The Server's OS kernel sends a FIN N packet to the Client: "I have sent everything; I am closing my direction now."

ACK N+1 Packet

- The Client receives FIN N and sends the final confirmation packet, ACK N+1, back to the Server.
- At this point, both directions of the transmission path are completely closed.

3.1. Socket Address Structures

IPv4 Socket Address Structures

```
struct in_addr {
    in_addr_t       s_addr;      /* 32-bit IPv4 address */
};

struct sockaddr_in {
    uint8_t         sin_len;     /* Structure length (16 bytes) */
    sa_family_t     sin_family;  /* Address family: always AF_INET */
    in_port_t       sin_port;    /* 16-bit port number (Network Byte Order) */
    struct in_addr  sin_addr;    /* 32-bit IP address (Network Byte Order) */
    char            sin_zero[8]; /* 8-byte padding array (unused, always set to 0) */
};
```

Generic Socket Address Structure

```
struct sockaddr {
    uint8_t      sa_len;       /* 1 byte: Length of structure (16 bytes) */
    sa_family_t  sa_family;    /* 1 or 2 bytes: Protocol family (AF_INET, AF_INET6...) */
    char         sa_data[14];  /* 14 bytes: Contains protocol-specific address */
};
```

IPv6 Socket Address Structure

```
struct in6_addr {
    uint8_t   s6_addr[16];        /* 128-bit IPv6 address (16 bytes) */
};

struct sockaddr_in6 {
    uint8_t         sin6_len;      /* 1 byte: Length of struct (28 bytes) */
    sa_family_t     sin6_family;   /* 1 or 2 bytes: Always AF_INET6 */
    in_port_t       sin6_port;     /* 2 bytes: Port number (Network Byte Order) */
    uint32_t        sin6_flowinfo; /* 4 bytes: Flow information */
    struct in6_addr sin6_addr;     /* 16 bytes: IPv6 address */
    uint32_t        sin6_scope_id; /* 4 bytes: Scope identifier (Scope ID) */
};
```

New Generic Socket Address Structure

```
struct sockaddr_storage {
    uint8_t      ss_len;     /* Structure length */
    sa_family_t  ss_family;  /* Address family: AF_INET, AF_INET6... */
};
```

This structure has a size of 128 bytes.

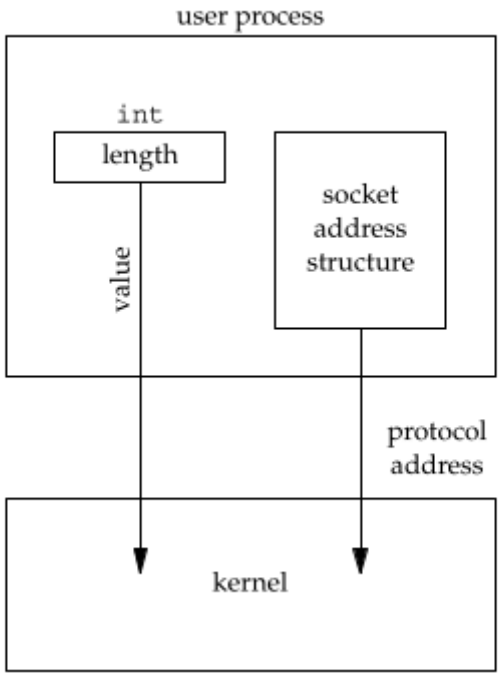
Advantages over struct sockaddr

- Large enough to store any type of address
- Strictest memory alignment: It is aligned on a memory boundary divisible by 64-bit (or 128-bit), enabling extremely fast CPU data access and absolute safety when performing pointer casting.

3.2. Value-Result Arguments

3.2.1. Direction - From Process Down to Kernel

Consists of 3 functions: bind(), connect(), sendto()



Right arrow: socket address structure -> Pass protocol address

- Pass a pointer pointing to the location where the address structure is stored in RAM (e.g., the variable servaddr containing IP and Port)
- The Kernel relies on this pointer to identify the starting memory address to read

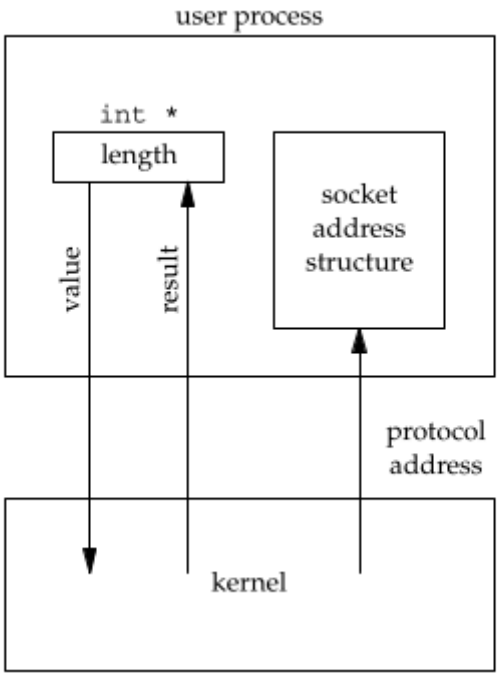
Left arrow: int length -> Pass value

- Pass the size and length of that structure (e.g., sizeof(servaddr) which is 16)
- The Kernel reads this value to know how many bytes it needs to copy

```
struct sockaddr_in serv;
connect(sockfd, (SA *)&serv, sizeof(serv));
```

3.2.2. Direction - From Kernel Up to Process

Consists of 4 functions: accept(), recvfrom(), getsockname(), getpeername().




```
struct sockaddr_storage cli;
socklen_t len;

len = sizeof(cli);                      /* 1. len acts as a VALUE */
accept(listenfd, (SA *)&cli, &len);      /* 2. Pass POINTER &len */
/* After the function finishes: len acts as the RESULT */
```

Inbound direction (Acts as Value)

- Assign len = sizeof(cli) (e.g., 128 bytes), then pass the address &len to the Kernel

Outbound direction (Acts as Result)

- The Kernel writes the Client's information into cli
- Assuming an IPv4 network connection (16 bytes)

- The Kernel overwrites the value in variable len with 16
- When the accept function returns, printing len yields 16 => A single parameter that both carries a value when passed in and receives a result when returning -> It is called a Value-Result Argument

3.3. Byte Ordering Functions

3.3.1. What is Endianness?

Example: hex number 0x0102

- Byte 0x01 is the Most Significant Byte (MSB)
- Byte 0x02 is the Least Significant Byte (LSB)

When storing these 2 bytes in 2 consecutive RAM memory locations (memory address A and A+1), systems typically divide them as follows:

BIG-ENDIAN

- Memory address A : 0x01 (Most significant byte first)
- Memory address A+1 : 0x02

LITTLE-ENDIAN

- Memory address A : 0x02 (Least significant byte first)
- Memory address A+1 : 0x01

```
int main(void) {
    union {
        short s;
        char  c[sizeof(short)];
    } un;
    un.s = 0x0102;

    printf("Size of short type: %zu bytes\n", sizeof(short));

    printf(" - Byte at first address (c[0]): 0x%02x (%d)\n", (unsigned char)un.c[0], un.c[0]);
    printf(" - Byte at next address (c[1]): 0x%02x (%d)\n", (unsigned char)un.c[1], un.c[1]);
    if (un.c[0] == 1 && un.c[1] == 2) {
        printf("BIG-ENDIAN \n");
    } else if (un.c[0] == 2 && un.c[1] == 1) {
        printf("LITTLE-ENDIAN\n");
```

```
    } else {
        printf("Unknown\n");
    }
    return 0;
}
```

Output

```
Size of short type: 2 bytes
Byte at first address (c[0]): 0x02 (2)
Byte at next address (c[1]): 0x01 (1)
LITTLE-ENDIAN
```

3.3.2. Conversion Functions

```
#include <netinet/in.h>

uint16_t htons(uint16_t host16bitvalue);  /* Host to Network Short */
uint32_t htonl(uint32_t host32bitvalue);  /* Host to Network Long  */

uint16_t ntohs(uint16_t net16bitvalue);   /* Network to Host Short */
uint32_t ntohl(uint32_t net32bitvalue);   /* Network to Host Long  */
```

When sending (Host to network)

- servaddr.sin_port = htons(80); -> Convert port from host to network byte order.
- servaddr.sin_addr.s_addr = htonl(INADDR_ANY); -> Convert IP from host to network byte order.

When receiving (Network to host)

- Reading the port sent up by the client: int port = ntohs(cliaddr.sin_port); -> Convert back from network to host byte order to print on the screen.

```
void print_bytes(const char *label, unsigned char *ptr, int size)
{
    printf("%-20s: ", label);
    for (int i = 0; i < size; i++)
    {
        printf("[0x%02x] ", ptr[i]);
    }
    printf("\n");
```

```
}
int main(void)
{
    uint16_t port = 0x1234;
    printf("--- 16-BIT (PORT) ---\n");
    printf("Original value      : 0x%04x (%u)\n", port, port);
    print_bytes("Host (Local RAM)", (unsigned char *)&port, sizeof(port));
    uint16_t net_port = htons(port);
    print_bytes("Network (htons)", (unsigned char *)&net_port, sizeof(net_port));
    uint16_t back_port = ntohs(net_port);
    printf("Back to Host (ntohs): 0x%04x (%u)\n\n", back_port, back_port);
    uint32_t val = 0x12345678;
    printf("--- 32-BIT (IP) ---\n");
    printf("Original value      : 0x%08x\n", val);
    print_bytes("Host (Local RAM)", (unsigned char *)&val, sizeof(val));
    uint32_t net_val = htonl(val);
    print_bytes("Network (htonl)", (unsigned char *)&net_val, sizeof(net_val));
    uint32_t back_val = ntohl(net_val);
    printf("Back to Host (ntohl): 0x%08x\n", back_val);
    return 0;
}
```

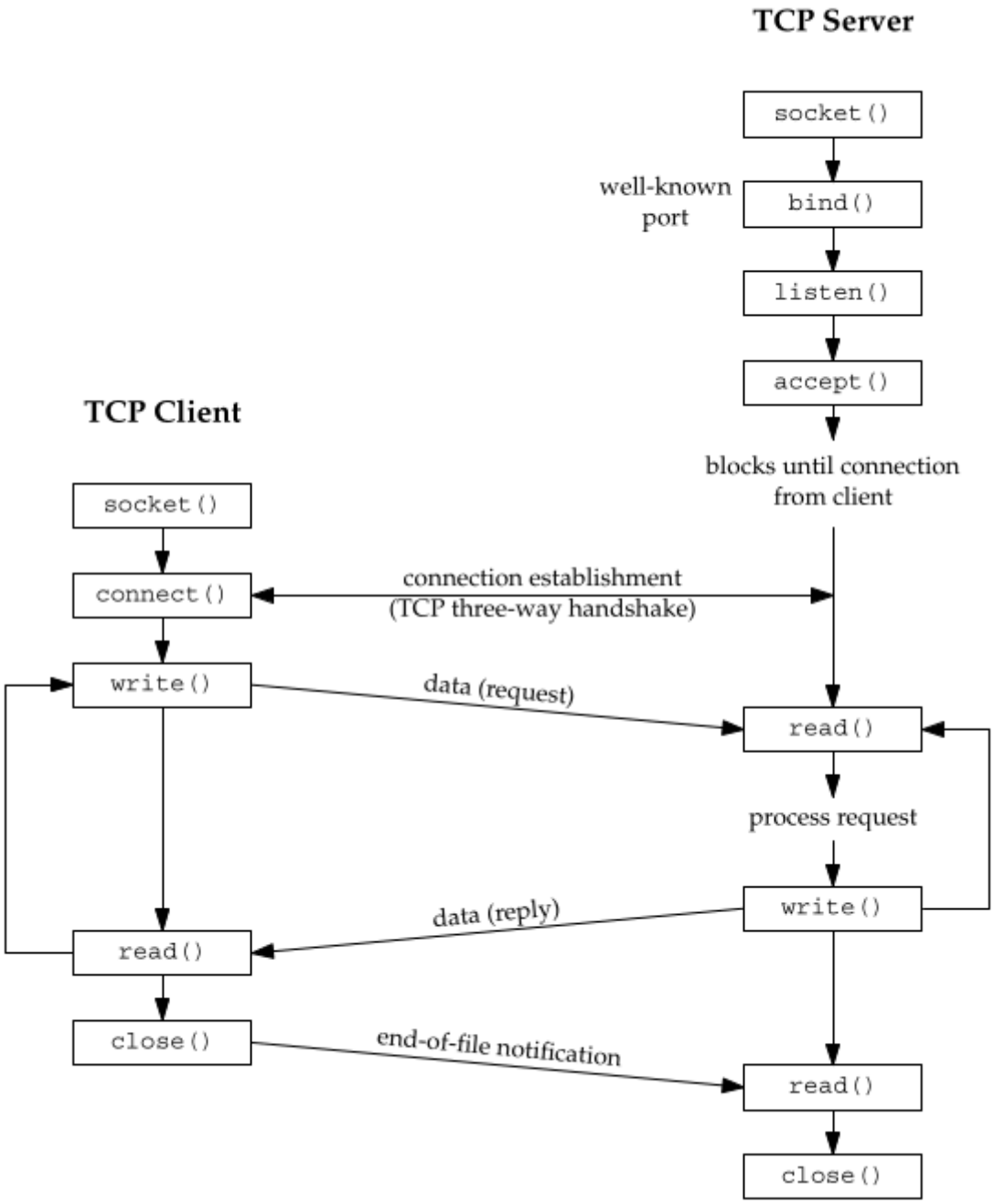
Output

```
--- 16-BIT (PORT) ---
Original value      : 0x1234 (4660)
Host (Local RAM)    : [0x34] [0x12]
Network (htons)     : [0x12] [0x34]
Back to Host (ntohs): 0x1234 (4660)

--- 32-BIT (IP) ---
Original value      : 0x12345678
Host (Local RAM)    : [0x78] [0x56] [0x34] [0x12]
Network (htonl)     : [0x12] [0x34] [0x56] [0x78]
Back to Host (ntohl): 0x12345678
```

4.1. Socket Function

TCP Client-Server Flow Diagram



- The server side runs first, executing the commands sequentially: socket() -> bind() -> listen() -> accept()
- The server blocks at the accept() function until a connect() call is issued from the Client
- The Client's connect() call automatically initiates the TCP 3-way handshake
- When the Client calls close(), TCP sends a FIN flag. When the Server side receives a return value of 0 (EOF) via read() -> The Server recognizes that the Client has disconnected and calls close() to clean up

The connect() function is used by the TCP Client to actively establish a connection to the TCP Server.

The Client does not need to call the bind() function before connect(). The Operating System (Kernel) will automatically assign an available random port (Ephemeral Port) and select the most

appropriate network interface IP address for the Client

Errors when calling connect()

ETIMEDOUT Error (Connection timed out): A SYN packet is sent, but no response is received -> The Kernel does not give up immediately; instead, it retransmits multiple times. If no response is received after about 75 seconds -> It returns the ETIMEDOUT error.

ECONNREFUSED Error (Connection refused): Caused when no Server is listening on that port -> Returns a packet with the RST flag set.

EHOSTUNREACH / ENETUNREACH Error (No route to host / Network unreachable)

- An intermediate Router along the transmission path finds no route to the destination IP subnet and immediately sends back an ICMP "Destination Unreachable" packet.
- The Client still retransmits the SYN packet multiple times; if it remains unsuccessful after a period of time, it reports the error to the screen.

When connect() fails, that socket is considered completely unusable and cannot be reused.

```
/* Incorrect */
if (connect(sockfd, &addr1, sizeof(addr1)) < 0) {
    connect(sockfd, &addr2, sizeof(addr2));
}

/* Correct */
if (connect(sockfd, &addr1, sizeof(addr1)) < 0) {
    close(sockfd);
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    connect(sockfd, &addr2, sizeof(addr2));
}
```

4.2. The bind() Function

Binds a local IP address and a Port number to the Socket - Used for Servers

- Client: The OS automatically assigns a random port -> No need to call bind()
- Server: Requires a fixed address (e.g., Port 80 for Web, Port 22 for SSH...) so Clients know which port to connect to

```
#include <sys/socket.h>
```

```
int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);
```

Wildcard (INADDR_ANY)

Using this option means the Server accepts connections on any IP address, as long as the port matches.

```
servaddr.sin_addr.s_addr = htonl(INADDR_ANY); /* IPv4 Wildcard */
serv.sin6_addr = in6addr_any; /* IPv6 Wildcard */
```

What if Port 0 is passed?

- The OS will assign a random Port.
- To discover that assigned random Port -> Use the getsockname() function (because bind() does not overwrite the port number back into the struct due to the const keyword).

Error: bind error: Address already in use (EADDRINUSE)

- Occurs because another program on the machine is currently using that Port.
- Terminating the Server with Ctrl+C and restarting it immediately -> The OS still holds that Port in the TIME_WAIT safety state for about 1–2 minutes.

4.3. The listen() Function

Used exclusively by TCP Servers, called after bind()

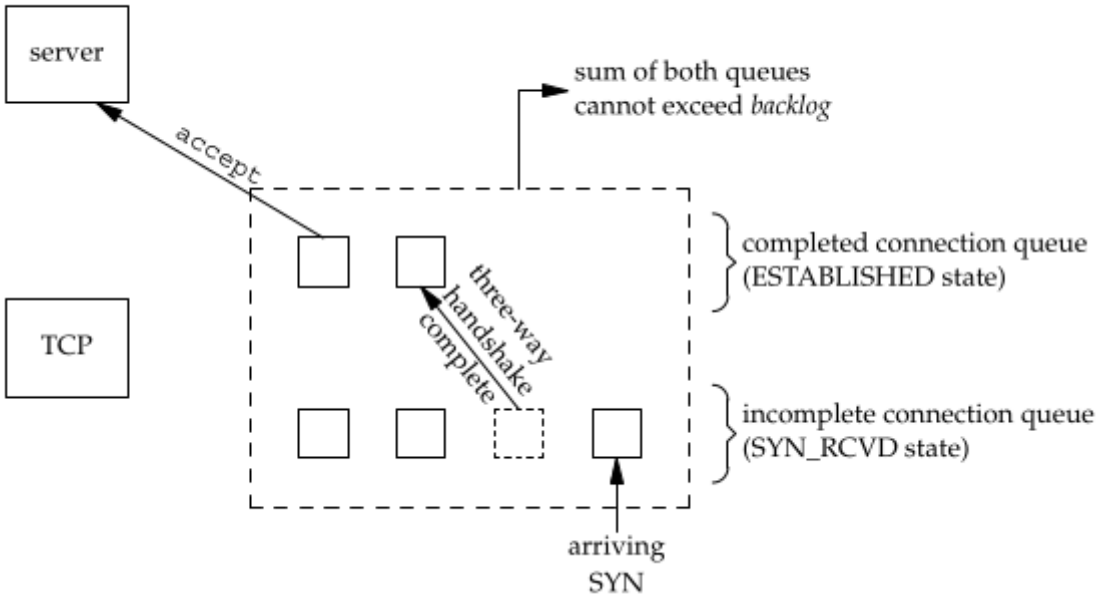
Responsibilities

Converts a Socket from an active socket to a passive socket to listen for incoming Clients

Specifies the size of the pending connection queue (backlog)

- The backlog parameter specifies the maximum number of connections the OS is allowed to hold in the queue before the Server can process them

The two-queue mechanism in the Kernel



Incomplete connection queue (SYN Queue)

- Contains Clients that have only sent the initial SYN packet
- The Server has responded with SYN + ACK and is waiting for the Client's final ACK packet to complete the handshake

Completed connection queue (Accept Queue)

- Contains connections that have fully completed the 3-way handshake
- These connections wait for the Server to call accept()
- If this queue is empty -> accept() blocks until a new connection arrives

Meaning of backlog: Represents the maximum capacity of the queues

What happens when the queue is full?: TCP silently ignores incoming SYN packets without sending a rejection packet (RST)

- Queues are usually full only temporarily, and space will free up in a few milliseconds
- If rejected -> The Client immediately receives a "Connection refused" error
- If ignored -> The Client assumes the packet was dropped in transit and retransmits it after a few seconds
  -> By then, the Server has cleared some space and can accept the Client

4.4. The accept() Function

After the 3-way handshake is complete -> The Client is moved to the Completed connection queue

-> The Server's accept() function retrieves the Client from that queue; if the queue is empty, accept() blocks to wait for a Client

```
#include <sys/socket.h>


int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

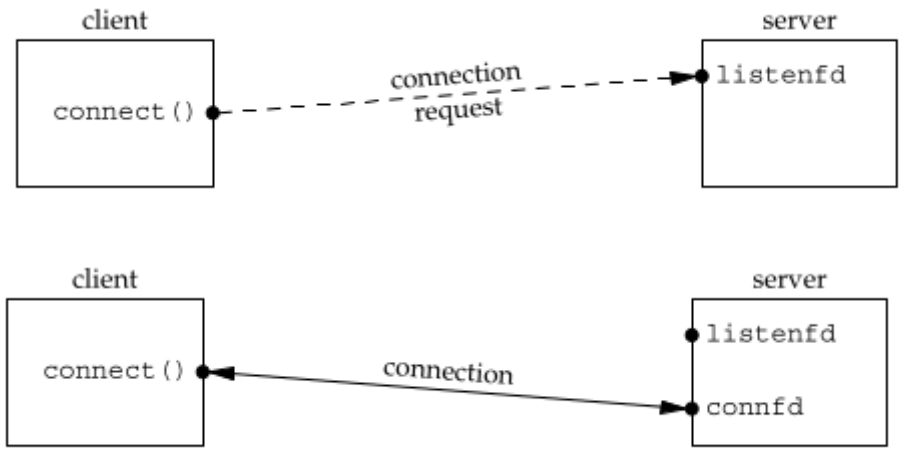
Listening Socket (listenfd)

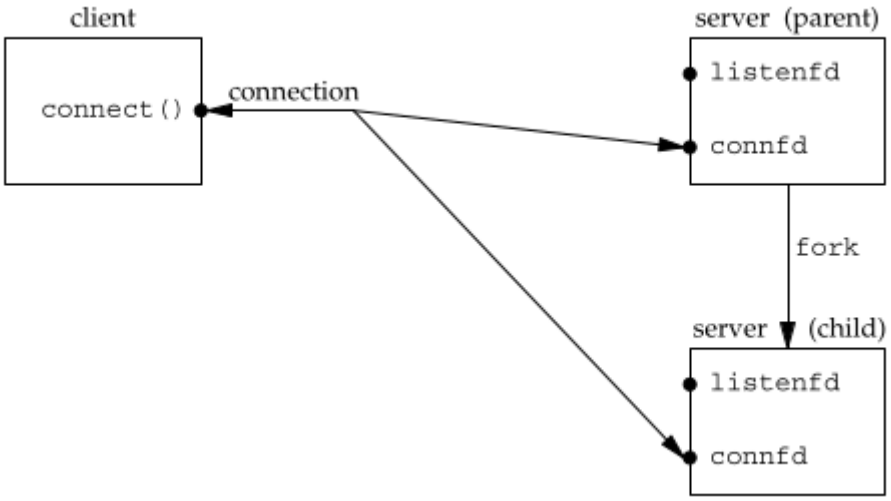
- Created using the socket() function
- Acts as a guard to listen for connections (bind, listen, accept)
- Only a single instance exists throughout the lifetime of the Server; it terminates only when the Server shuts down

Connected Socket (connfd)

- Automatically generated by the accept() function
- Communicates directly with the Client (read, write)
- Each Client gets its own dedicated socket
- Once done with a Client, close that specific socket via close(connfd)

5. Concurrent Server





Before connection is accepted

- Server blocks at the accept() function
- Client calls connect() to send a request to the Server's listenfd to perform the 3-way handshake

Immediately after accept()

- Handshake succeeds -> The OS creates a new socket called connfd
- The TCP connection is established between the Client and the Server's connfd

After fork()

- The child process is created and inherits copies of descriptors from the parent
- Both parent and child hold listenfd, and both point to the same connfd

After cleanup

- Child calls close(listenfd): Removes the reference to the listening socket (it no longer accepts new Clients)
- Parent calls close(connfd): Removes the reference to the Client's connfd socket that the child is now handling

-> The child process handles communication with the Client over connfd, while the parent keeps only listenfd and returns to call accept() for the next Client

Lab 1

Server

```
#define PORT 9877
#define MAX  4096
```

```
void str_echo(int sockfd)
{
    char buf[MAX];
    int n;
    while ((n = read(sockfd, buf, sizeof(buf))) > 0)
    {
        write(sockfd, buf, n);
    }
}
int main()
{
    int listenfd, connfd;
    struct sockaddr_in servaddr = {0}, cliaddr;
    socklen_t clilen = sizeof(cliaddr);
    listenfd = socket(AF_INET, SOCK_STREAM, 0);
    servaddr.sin_family      = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port        = htons(PORT);
    if (bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
    {
        perror("bind error");
        exit(1);
    }
    if (listen(listenfd, 1024) < 0)
    {
        perror("listen error");
        exit(1);
    }
    printf("Server listening on port %d...\n", PORT);
    while (1)
    {
        clilen = sizeof(cliaddr);
        connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &clilen);
        if (connfd < 0) continue;
        printf("New connection: %s:%d\n", inet_ntoa(cliaddr.sin_addr),
ntohs(cliaddr.sin_port));
        if (fork() == 0)
        {
            close(listenfd);
            str_echo(connfd);
```

```
            close(connfd);
            exit(0);
        }
        close(connfd);
    }
    return 0;
}
```

Client

```
#define PORT 9877
#define MAX  4096
int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr = {0};
    char sendline[MAX], recvline[MAX];
    char *ip = (argc > 1) ? argv[1] : "127.0.0.1";
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    servaddr.sin_family = AF_INET;
    servaddr.sin_port   = htons(PORT);
    inet_pton(AF_INET, ip, &servaddr.sin_addr);
    if (connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
    {
        perror("connect error");
        exit(1);
    }
    printf("Connected to %s:%d. Enter message (Ctrl+D to quit):\n", ip, PORT);
    while (fgets(sendline, MAX, stdin) != NULL) {
        write(sockfd, sendline, strlen(sendline));
        int n = read(sockfd, recvline, sizeof(recvline) - 1);
        if (n <= 0)
        {
            printf("Server disconnected!\n");
            break;
        }
        recvline[n] = '\0';
        printf("Server reply: %s", recvline);
    }
    close(sockfd);
```

```
    return 0;
}
```

Output

```
Server listening on port 9877...
New connection: 127.0.0.1:50798
```

```
$ ./client 127.0.0.1
Connected to 127.0.0.1:9877. Enter message (Ctrl+D to quit):
from client 1
Server reply: from client 1
```

6. close / getsockname / getpeername Functions1

6.1. close() Function

The close() function: Only decrements the reference count (ref_count). If another process still holds that socket (ref_count > 0), the TCP connection is not closed yet and the FIN packet is not sent.

```
#include <unistd.h>
int close(int sockfd);
```

At the Application Layer: The socket is marked as closed, and performing read() or write() operations on that socket is no longer permitted.

At the Network Layer (TCP Kernel): The OS will check:

- If un-sent data remains in the send buffer -> TCP will attempt to transmit all remaining data to the peer.
- Afterwards, TCP begins sending the FIN packet to perform the 4-way teardown handshake.

(This behavior can be modified using SO_LINGER if you want to close immediately and discard pending data.)

What if the Parent does not call close(connfd)?

```
while (1)
{
    connfd = accept(...);
    if (fork() == 0)
    {
        /* Child handles connection...*/
        close(connfd);
        exit(0);
    }
    close(connfd); / IF THE PARENT FORGETS THIS LINE*/
}
```

File Descriptor Exhaustion (Resource Depletion)

- Each process in Linux has a limit on the number of files it can open simultaneously (default is typically 1024).

-> Results in the "Too many open files" (EMFILE) error.

Client Connection Hangs

- When the Child process finishes handling the request, it calls close(connfd) -> ref_count decrements from 2 down to 1.
- Because the Parent never called close(), ref_count remains permanently stuck at 1.
- The Operating System sees that ref_count is still greater than 0; therefore, the Kernel never sends the FIN packet to the Client.

6.2. getsockname and getpeername Functions

getsockname Function

```
int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
```

When to use?

- Client does not call bind(): Most TCP Clients do not explicitly select a port; instead, the Kernel automatically assigns a random port. After connect() succeeds, to check the Client's IP and port -> call getsockname()
- Server binds to Port 0: The Kernel automatically selects an available port; to discover which port was assigned -> call getsockname()
- Checking address family (IPv4 or IPv6)

getpeername Function

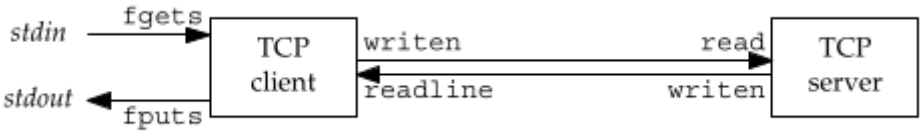
Queries the OS to retrieve the peer's information: IP and Port?

Use when a child process executes exec() -> the child process loses all variable state/information -> calls getpeername()

7. TCP Echo Client/Server

7.1. Echo Client/Server Model

Whatever data the Client sends, the Server echoes back identical data.



- Client: The user inputs data via the fgets function -> The Client sends the data over to the Server using the writen function.
- Server: Reads data from the socket via the read function and sends the data back to the Client via the writen function.
- Client: Reads the response from the Server via readline -> Outputs data using fputs.
- The two arrows between Client and Server in the diagram actually share a single full-duplex TCP connection, meaning data can be transmitted and received simultaneously over the same transmission path.

7.2. main() Function of TCP Echo Server

Creating the Socket and setting up Server address

```
listenfd = Socket(AF_INET, SOCK_STREAM, 0);

bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family      = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port        = htons(SERV_PORT);

Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));
Listen(listenfd, LISTENQ);
```

Waiting for Client connections

```
for ( ; ; ) {
    clilen = sizeof(cliaddr);
    connfd = Accept(listenfd, (SA *) &cliaddr, &clilen);
```

The Accept() function blocks execution until a Client successfully completes the 3-way handshake.

Once the handshake is complete, Accept() returns the connfd connection descriptor and populates cliaddr with the Client's IP and Port.

Handling multiple Clients using fork()

```
if ( (childpid = Fork()) == 0 )
{
        Close(listenfd);              /* Child does not accept new connections -> close
listenfd */
        str_echo(connfd);             /* Child serves the client: echo back whatever is read
*/
        exit(0);                      /* Task complete -> terminate child process */
    }
    Close(connfd);                    /* Parent process closes connfd, loops back to accept
new clients */
}
```

7.2. The str_echo Function

```
void str_echo(int sockfd)
{
    char buf[MAX];
    int n;
    while ((n = read(sockfd, buf, sizeof(buf))) > 0) {
        write(sockfd, buf, n);
    }
}
```

The read(sockfd, buf, sizeof(buf)) function blocks, waiting to read a block of data sent by the client into the buf array.

The variable n stores the actual number of bytes read:

- If the number of bytes read is greater than 0 (n > 0): The Server calls write(sockfd, buf, n) to send those exact n bytes back to the Client.
- The while loop continues waiting for subsequent messages.

When does the while loop terminate?

- When the Client disconnects, the Client-side OS sends a FIN packet to the Server.
- When the Server receives the FIN packet, read() returns 0, which is equivalent to receiving an EOF (End-Of-File) signal.
- Since n = 0, the while loop terminate

7.3. TCP Echo Client: main Function

argv[1] (Receives Server IP from the command line)

- When running the client from the terminal
- argv[1] is the IP address string of the Server to which the client wants to connect

inet_pton()

- Presentation to Numeric (Converts human-readable IP strings into 32-bit binary network formats that the network interface card understands)
- The result is written directly into servaddr.sin_addr

connect()

- The system call that triggers the TCP 3-way handshake process with the Server

str_cli(stdin, sockfd): Calls the Client processing function

- Reads input from the keyboard (stdin) -> Sends it through sockfd to the Server -> Receives the response and prints it to the screen

```
int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr;
    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");
    sockfd = Socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port   = htons(SERV_PORT);
    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
    Connect(sockfd, (SA *) &servaddr, sizeof(servaddr));
    str_cli(stdin, sockfd);
    exit(0);
}
```

7.4. TCP Echo Client: str_cli Function

```
void str_cli(FILE *fp, int sockfd)
{
    char sendline[MAXLINE], recvline[MAXLINE];
    while (Fgets(sendline, MAXLINE, fp) != NULL)
    {
        Writen(sockfd, sendline, strlen(sendline));
        if (Readline(sockfd, recvline, MAXLINE) == 0)
            err_quit("str_cli: server terminated prematurely");
        Fputs(recvline, stdout);
    }
}
```

The while loop terminates when fgets() returns NULL

- Ctrl+D -> EOF signal -> Returns NULL
- The while loop stops -> The str_cli() function terminates
- Program control returns to main() -> Executes exit(0)
- Upon exit(0), the OS automatically closes sockfd and sends a FIN packet to the Server

8.Normal Termination, SIGPIPE Signal, server/client crash scenarios

8.1. Normal Termination

When the user presses Ctrl+D -> the fgets() function on the Client returns NULL -> the Client calls exit(0)

Upon exit(0), the OS closes the Client's socket -> Sends a FIN packet to the Server -> The Server sends back an ACK

On the Child Server, the read function receives the FIN -> returns 0 and exits the str_echo function

The Child Server also calls exit(0) -> The OS closes the Child Server's socket -> Sends a FIN packet to the Client -> The Client replies with an ACK

- TCP connection is fully closed
- The Client enters the TIME_WAIT state (retaining the port for an additional period to ensure the ACK packet reaches its destination safely)
-

Lab 2

Server crash - kill server child process

```
$ ./client 127.0.0.1
Connected to 127.0.0.1:9877. Enter message (Ctrl+D to quit):
from client 1
Server disconnected!
```

Client crash

When the Client process dies suddenly, the operating system Kernel automatically closes the client's socket and sends a FIN packet to the Server.

The Child Server process (fork) waiting at the read() call receives this FIN packet -> read() returns 0 (EOF).

The Server's str_echo function exits the while loop, closes connfd, and calls exit(0) to terminate gracefully.

-> The Server does not hang. *Correct handling of SIGPIPE*

```
signal(SIGPIPE, SIG_IGN);
```

```
if (write(sockfd, sendline, strlen(sendline)) < 0)
{
    if (errno == EPIPE)
    {
        printf("Error: Broken pipe (SIGPIPE ignored, peer closed socket)!\n");
    } else {
        perror("write error");
    }
    break;
}
```

Output

```
Connected to 127.0.0.1:9877. Enter message (Ctrl+D to quit):
hh

>>> [SIGNAL TRIGGERED] Caught SIGPIPE (signal 13)!
>>> [ERROR] Second write failed with EPIPE (Broken pipe)!
```

8.2. SIGPIPE Signal

The Child Server crashes without the Client knowing, and the Client continues sending data

First write() call: Data is sent to the Server. Because the Server process is dead, the Server's Kernel immediately responds with a RST (Reset) packet stating: "No process is listening here anymore!".

Second write() call: The Client attempts to write data to a socket that has already received a RST packet -> The Client's Operating System delivers a SIGPIPE signal to the Client process.

Consequences of SIGPIPE

The default OS action for SIGPIPE is to terminate the process immediately -> Broken pipe

Ignoring the SIGPIPE signal

```
signal(SIGPIPE, SIG_IGN); /* Ignore the SIGPIPE signal */
```

The program will not crash, and the write() function will return -1 with errno set to EPIPE.

8.3. Crashing of Server Host

This is an issue at the hardware or network infrastructure level (such as a server power outage, etc.)

The Client calls write() to send data and calls readline() to wait for data to be returned

TCP Retransmission mechanism

- The core nature of TCP is reliable transmission. When sending data without receiving an ACK packet back, TCP assumes the packet was lost and retransmits it
- Client TCP will continuously retransmit the packet approximately 12 times
- If no ACK packet is returned after about 9 minutes of retransmissions -> The readline() function returns the ETIMEDOUT error (Connection timed out)

Solutions

Method 1: Set a shorter timeout (e.g., if the server does not respond within ~5 seconds -> report an error immediately)

Method 2: Enable the SO_KEEPALIVE socket option

8.4. Crashing and Rebooting of Server Host

While the Server and Client are connected, the Server crashes and reboots

- While the Server is rebooting, the Client is unaware because no packets are sent (if SO_KEEPALIVE is not enabled)

The Client continues to send data; by this time, the Server has booted back up and connected to the network

- All TCP connection state information is stored in RAM -> RAM has now been cleared
- The Server currently has no record of a connection with that Client and no 3-way handshake was performed -> Sends back a RST packet
- The Client's read/write call returns -1 with the error "Connection reset by peer"

9. Elementary UDP Sockets

9.1. recvfrom and sendto Functions

```
#include <sys/socket.h>

/* Receive data (read packet) */
ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags,
                 struct sockaddr *from, socklen_t *addrlen);

/* Send data (transmit packet) */
ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags,
               const struct sockaddr *to, socklen_t addrlen);
```

Success: returns the actual number of bytes read/written

Failure: returns -1

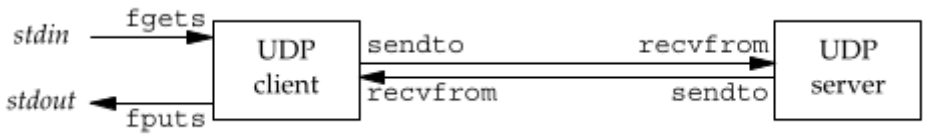
Zero-byte datagrams (return 0)

In TCP: When the read() function returns 0, it indicates that the remote peer has closed the connection (EOF/FIN).

In UDP:

- UDP is connectionless, so there is no concept of "closing a connection."
- UDP permits sending a 0-byte payload packet (the packet in this case consists only of a 20-byte IP Header + an 8-byte UDP Header, with an empty payload).
- Therefore, when recvfrom() returns 0, it is completely normal — it simply means the peer sent an empty packet, not that the peer has disconnected.

9.2. UDP Echo Server: main Function



The Client reads a line of text entered by the user from the keyboard (stdin) using the fgets() function.

The Client packages this string into a datagram and sends it to the Server using the sendto() function.

The Server waits at the recvfrom() function; upon receiving the packet, it extracts the data along with the client's address.

The Server uses the sendto() function to echo that exact data back to the client's address.

The Client waits at recvfrom() to receive the response, then prints it to the screen (stdout) using the fputs() function.

```
#include "unp.h"

int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;

    sockfd = Socket(AF_INET, SOCK_DGRAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family      = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port        = htons(SERV_PORT);

    Bind(sockfd, (SA *) &servaddr, sizeof(servaddr));

    dg_echo(sockfd, (SA *) &cliaddr, sizeof(cliaddr));
}
```

9.3. UDP Echo Server: dg_echo Function

```
void dg_echo(int sockfd, SA *pcliaddr, socklen_t clilen)
{
    int n;
```

```
    socklen_t len;
    char mesg[MAXLINE];

    for ( ; ; ) {
        len = clilen;  /* Must re-assign before EVERY recvfrom call*/
        n = Recvfrom(sockfd, mesg, MAXLINE, 0, pcliaddr, &len);

        Sendto(sockfd, mesg, n, 0, pcliaddr, len);
    }
}
```

The &len parameter passed into recvfrom is a Value-Result argument. After the first execution, the value of len may change. Therefore, before entering the next recvfrom call, you must re-assign the initial maximum buffer length (clilen) back to len.

Characteristics of UDP Server

This function does not terminate

- In TCP, when a client closes the connection, the read() function returns 0 (EOF) -> The server exits the loop and terminates the child process.
- With UDP, there is no concept of a connection, so EOF never occurs. The server runs an infinite for (; ;) loop unless the process is killed.

Iterative Server vs. Concurrent Server

TCP Server

- To serve multiple clients simultaneously, the parent server must call fork() to create separate child processes for each client.
- Each connection has its own dedicated socket and receive buffer.

UDP Server

- It is an Iterative Server (processes sequentially).
- It uses only 1 process and 1 socket. All clients send packets to this single shared socket. The server fetches and processes one packet at a time, sending back replies sequentially.

UDP Queue (Socket Receive Buffer & FIFO)

- The OS UDP layer reserves a dedicated socket receive buffer for that socket.
- If multiple clients send packets simultaneously while the server is busy processing, the packets wait in the buffer queue following the FIFO (First-In, First-Out) principle.
- If incoming packets arrive too quickly and overflow this buffer, the operating system silently drops any subsequent incoming packets (since UDP lacks congestion control

mechanisms).

Protocol-independent nature

- The main function (Figure 8.3) is protocol-dependent: it initializes a socket specifically for IPv4 (AF_INET, struct sockaddr_in).
- The dg_echo function (Figure 8.4) is protocol-independent: it only accepts a generic pointer type SA * (struct sockaddr *). This means the function can be reused completely as-is for IPv6 without modifying a single line of code inside it

9.4. UDP Echo Client: main Function

```
#include "unp.h"

 int main(int argc, char **argv)
 {
     int sockfd;
     struct sockaddr_in servaddr;

     if (argc != 2)
         err_quit("usage: udpcli <IPaddress>");

     bzero(&servaddr, sizeof(servaddr));
     servaddr.sin_family = AF_INET;
     servaddr.sin_port   = htons(SERV_PORT);
     Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);

     sockfd = Socket(AF_INET, SOCK_DGRAM, 0);

     dg_cli(stdin, sockfd, (SA *) &servaddr, sizeof(servaddr));

     exit(0);
 }
```

9.5. UDP Echo Client: dg_cli Function

```
void dg_cli(FILE *fp, int sockfd, const SA *pservaddr, socklen_t servlen)
{
    int n;
    char sendline[MAXLINE], recvline[MAXLINE + 1];
```

```
    while (Fgets(sendline, MAXLINE, fp) != NULL) {

        Sendto(sockfd, sendline, strlen(sendline), 0, pservaddr, servlen);

        n = Recvfrom(sockfd, recvline, MAXLINE, 0, NULL, NULL);

        recvline[n] = 0;    /* Thêm ký tự k□t thúc chu□i '\0' */
        Fputs(recvline, stdout);
    }
}
```

Characteristics

When is a Port assigned to the Client? (Ephemeral Port)?

- With TCP: The operating system automatically assigns a random port as soon as connect() is called.
- With UDP: Since there is no connect(), the operating system automatically assigns a random port (ephemeral port) to the Client's socket on the very first sendto() call.

Security risks from NULL, NULL in recvfrom: Passing NULL, NULL for the last two parameters means: "The Client only needs to receive data and does not care who sent it." Any rogue process or attacker on the network who knows the Client's IP and Port can inject junk/spoofed packets into this socket.

Protocol independence: Runs on both IPv4 and IPv6 seamlessly.

Lab 3

Server

```
#define PORT 9877
#define MAX  4096

int main()
{
    int sockfd;
    struct sockaddr_in servaddr = {0}, cliaddr;
    socklen_t clilen;
    char mesg[MAX];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
```

```
        perror("socket error");
        exit(1);
    }

    servaddr.sin_family      = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port        = htons(PORT);

    if (bind(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
    {
        perror("bind error");
        exit(1);
    }

    printf("UDP Server listening on port %d...\n", PORT);

    while (1)
    {
        clilen = sizeof(cliaddr);
        int n = recvfrom(sockfd, mesg, sizeof(mesg), 0, (struct sockaddr *)&cliaddr, &clilen);
        if (n < 0)
        {
            perror("recvfrom error");
            continue;
        }

        printf("Received %d bytes from %s:%d\n", n, inet_ntoa(cliaddr.sin_addr),
ntohs(cliaddr.sin_port));

        sendto(sockfd, mesg, n, 0, (struct sockaddr *)&cliaddr, clilen);
    }

    close(sockfd);
    return 0;
}
```

Client

```
#define PORT 9877
#define MAX  4096
```

```
int main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr = {0};
    char sendline[MAX], recvline[MAX];
    char *ip = (argc > 1) ? argv[1] : "127.0.0.1";

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0)
    {
        perror("socket error");
        exit(1);
    }

    servaddr.sin_family = AF_INET;
    servaddr.sin_port   = htons(PORT);
    inet_pton(AF_INET, ip, &servaddr.sin_addr);

    printf("UDP Client ready. Sending to %s:%d. Enter message (Ctrl+D to quit):\n", ip, PORT);

    while (fgets(sendline, MAX, stdin) != NULL)
    {
        sendto(sockfd, sendline, strlen(sendline), 0, (struct sockaddr *)&servaddr,
sizeof(servaddr));

        int n = recvfrom(sockfd, recvline, sizeof(recvline) - 1, 0, NULL, NULL);
        if (n < 0) {
            perror("recvfrom error");
            break;
        }

        recvline[n] = '\0';
        printf("Server reply: %s", recvline);
    }

    close(sockfd);
    return 0;
}
```

No connect() required; once the socket is created, messages can be sent immediately.

Uses recvfrom() and sendto() (every send/receive operation must explicitly specify the peer's IP and Port)

10. I/O Multiplexing

10.1. I/O Model

When an application calls a read command (read()/recvfrom()), it undergoes 2 phases:

- Phase 1: Waiting for data to be ready: packets travel from the network cable to the network card (NIC), the OS unwraps the packets and places them into the Kernel buffer.
- Phase 2: The Kernel copies data from its buffer to the user program's buffer (buf/mesg

Blocking I/O Model - The Default Model The application calls the recvfrom() function.

- Stage 1: If no packets have arrived yet, the operating system puts the process to sleep, yielding the CPU to other software.
- Stage 2: When a packet arrives, the Kernel keeps the process in a waiting state while copying the data to the application buffer.
- Once copying is complete, the recvfrom() function wakes up and returns the result.

Non blocking I/O

If data is not ready, return an error so the process can perform other tasks

The application calls the recvfrom() function.

- Stage 1: If no data is available yet, the kernel returns an error code like EWOULDBLOCK (or EAGAIN) so the program can continue running.
- Stage 2: When a packet arrives, the data is copied over, and the function returns the number of successfully transferred bytes.

Polling

When an application runs a continuous loop calling recvfrom() while waiting for data, this is called Polling.-> Drawback: CPU overhead increases drastically due to constant spinning.

10.2. I/O Multiplexing Model

Stage 1 (Waiting for data to be ready): The process calls select() and blocks at the select() function.

- When at least one socket has data available, select() wakes up the process and specifically identifies which socket is ready.

Stage 2 (Copying data): The process calls recvfrom() on that specific socket.

- Copy data from the Kernel buffer to the application.

10.3. select() Function

```
#include <sys/select.h>
#include <sys/time.h>

int select(int maxfdp1,
           fd_set *readset,
           fd_set *writeset,
           fd_set *exceptset,
           const struct timeval *timeout);
```

Return value

>0: The number of ready descriptors (currently having data available to read/write)

0: Timeout elapsed with no descriptors becoming ready.

-1: Error occurred (most commonly the EINTR error caused by an interrupted system signal).

Maximum wait time (timeout)

```
struct timeval {
    long tv_sec;  /* Seconds */
    long tv_usec; /* Microseconds (1/1,000,000 second) */
};
```

Wait indefinitely (timeout = NULL): Sleeps indefinitely until at least one descriptor becomes ready.

Wait for a specified duration (e.g., 10.5 seconds): Wakes up as soon as a descriptor becomes ready, or when 10.5 seconds elapse if no data arrives.

Non-blocking / Immediate return (tv_sec = 0 and tv_usec = 0): Checks the status once and returns immediately.

10.4. poll() Function

```
#include <poll.h>

int poll(struct pollfd *fdarray, unsigned long nfds, int timeout);
```

Advantages of poll() over select()

Solves the Value-Result argument issue of select()

- In select(), the kernel overwrites the bitset -> must call FD_ZERO() and FD_SET() again in every loop iteration.
- In poll(), the events field and revents field are completely separated -> only need to set events once at the beginning and do not need to re-initialize it after each loop iteration.

No 1,024 socket limit

- The select() function has a hard limit in the Kernel: it can only monitor a maximum of 1,024 file descriptors (FD_SETSIZE). Exceeding this limit causes buffer overflow.
- The poll() function accepts a dynamic array. Can use malloc() to allocate an array for 5,000 or 50,000 sockets as desired, with no fixed limit.

Lab 3

Using select()

```
#define PORT 9877
int main()
{
    int listenfd, connfd, maxfd;
    fd_set rset, allset;
    char buf[1024];
    struct sockaddr_in servaddr = {0};
    int opt = 1;
    listenfd = socket(AF_INET, SOCK_STREAM, 0);
    setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(PORT);
    bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    listen(listenfd, 10);
    printf("Chat Server listening on port %d...\n", PORT);
    FD_ZERO(&allset);
    FD_SET(listenfd, &allset);
    maxfd = listenfd;
    while (1)
    {
        rset = allset;
        select(maxfd + 1, &rset, NULL, NULL, NULL);
```

```
        for (int i = 0; i <= maxfd; i++) {
            if (!FD_ISSET(i, &rset)) continue;
            if (i == listenfd)
            {
                connfd = accept(listenfd, NULL, NULL);
                FD_SET(connfd, &allset);
                if (connfd > maxfd) maxfd = connfd;
                printf("New client connected (fd %d)\n", connfd);
            }
            else
            {
                int n = read(i, buf, sizeof(buf));
                if (n <= 0)
                {
                    close(i);
                    FD_CLR(i, &allset);
                    printf("Client disconnected (fd %d)\n", i);
                }
                else
                {
                    for (int j = 0; j <= maxfd; j++)
                    {
                        if (FD_ISSET(j, &allset) && j != listenfd && j != i)
                        {
                            write(j, buf, n);
                        }
                    }
                }
            }
        }
    }
    return 0;
}
```

Using poll()

```
#define PORT 9877
#define MAX 1024
int main()
{
```

```
int listenfd, connfd, num_clients = 1;
struct pollfd fds[MAX];
char buf[1024];
struct sockaddr_in servaddr = {0};
int opt = 1;
listenfd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(PORT);
bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
listen(listenfd, 10);
printf("Chat Server (poll) listening on port %d...\n", PORT);
fds[0].fd = listenfd;
fds[0].events = POLLIN;
while (1)
{
    poll(fds, num_clients, -1);
    if (fds[0].revents & POLLIN)
    {
        connfd = accept(listenfd, NULL, NULL);
        fds[num_clients].fd = connfd;
        fds[num_clients].events = POLLIN;
        num_clients++;
        printf("New client connected (fd %d)\n", connfd);
    }
    for (int i = 1; i < num_clients; i++)
    {
        if (!(fds[i].revents & POLLIN)) continue;
        int n = read(fds[i].fd, buf, sizeof(buf));
        if (n <= 0)
        {
            printf("Client disconnected (fd %d)\n", fds[i].fd);
            close(fds[i].fd);
            fds[i] = fds[num_clients - 1]; // Move last element to current spot
            num_clients--;
            i--;
        }
        else
        {
```

```
                for (int j = 1; j < num_clients; j++)
                {
                    if (j != i) write(fds[j].fd, buf, n);
                }
            }
        }
    }
    return 0;
}
```

select() was restricted to 1024 connections and required re-assigning the bitmask before every loop iteration.

poll() was later introduced to fix both of these flaws: it uses an array of structures with no hard limit on the number of connections and keeps the Input field (events) clearly separated from the Output field (revents).

11. Name & Address Conversion

11.1. DNS and Resource Records

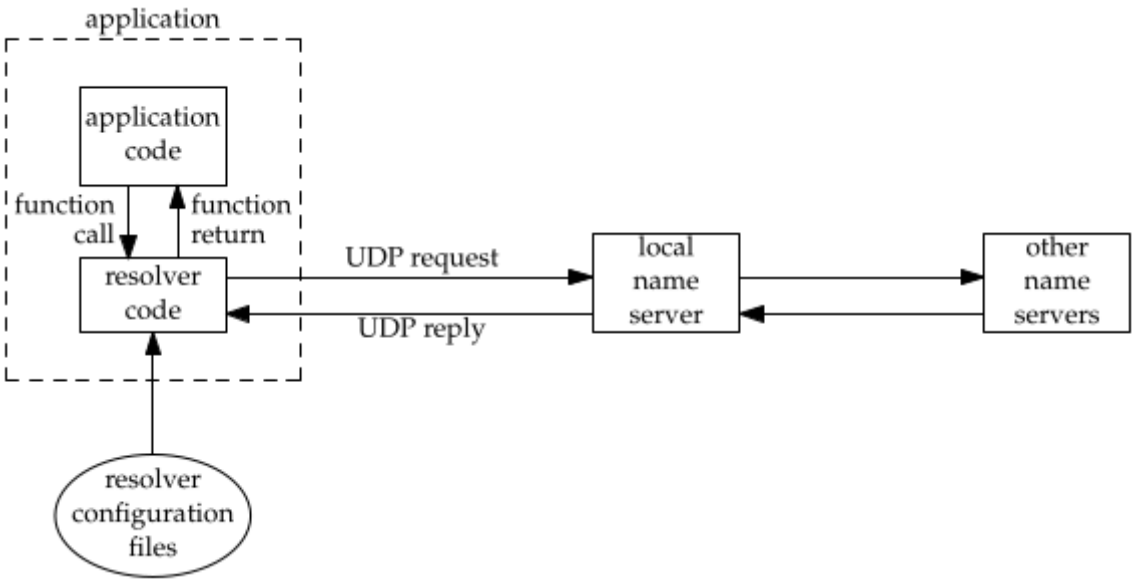
The DNS is used primarily to map between hostnames and IP addresses.

Why is Name and Address Conversion Needed?

A server may change its network provider or switch to a different IP address, while its domain name remains the same. **Resource Record Types**

- A (Address record): Maps a domain name to an IPv4 address.
- AAAA (Quad-A record): Maps a domain name to an IPv6 address.
- CNAME (Canonical Name): Alias record, maps a subdomain to a primary domain name.
- PTR (Pointer record): Reverse record, maps an IP address back to a domain name (used in reverse DNS).
- MX (Mail eXchanger): Specifies the mail server responsible for handling emails.

Domain Name Resolution Mechanism



Application code calls the getaddrinfo() function.

The Resolver library opens the /etc/resolv.conf file (resolver configuration file) to retrieve the IP address of the DNS server.

Sends a UDP request packet to port 53 of the local name server.

On the DNS server side:

- If the server has previously looked up this domain name -> It returns the result directly from its cache.
- If not available -> It contacts higher-level domain servers (other name servers) to query for the IP.

The local name server sends a UDP response packet (containing the resolved IP) back to the resolver (if the packet size exceeds 512 bytes -> it automatically switches to TCP).

The Resolver library converts the IP address into a network structure struct sockaddr_in and returns it to the application code.

11.2. Name and Address Conversion Functions

gethostbyname()

A function that resolves a hostname and queries DNS to return an IPv4 address

```
#include <netdb.h>


struct hostent *gethostbyname(const char *hostname);
/* Success: returns a pointer to a struct hostent*/
```

```
/* Failure: returns NULL and sets the error code in the global variable h_errno*/
```

struct hostent

Upon successful resolution, the function returns a pointer to a struct hostent structure.

```
struct hostent
{
    char  *h_name;       /* Official (canonical) name of the host */
    char **h_aliases;    /* Array of pointers to alias names (Aliases/CNAME) */
    int    h_addrtype;   /* Address type: always AF_INET for IPv4 */
    int    h_length;     /* Length of address: 4 bytes (for IPv4) */
    char **h_addr_list;  /* Array of pointers to the list of IP addresses, as a host can have
multiple IP addresses */
};
```

Revision #3
Created 2026-09-28 01:46:26 UTC by Huyen Nguyen
Updated 2026-09-29 10:36:36 UTC by Huyen Nguyen